

Name: Alok Kumar Tarini

Reg_no.: 2141020010

CA-Software

Assignment 2

Implementing Microservices and Event-Driven Architecture in an E-Commerce Platform

In a conceptual e-commerce application, a combination of Microservices Architecture and Event-Driven Architecture is adopted to streamline complex workflows such as order management, stock control, payment processing, and customer notifications.

Microservices Approach

- The system is decomposed into independent, modular services such as Order Service, Inventory Service, Payment Service, and Notification Service.
- These services interact through RESTful APIs, enabling seamless integration, independent scaling, and easier updates without affecting the entire system.
- This architecture supports fault isolation, improves maintainability, and enhances scalability.

Event-Driven Communication

- Services operate asynchronously by emitting and reacting to events.
- For instance, when a customer places an order, the Order Service emits an OrderPlaced event.
- The Inventory Service subscribes to this event to adjust stock levels accordingly.
- Simultaneously, the Notification Service listens to notify users via SMS or email.

Application of SOLID Principles

- **Single Responsibility (S):** Each microservice is responsible for a specific function within the platform.
- **Open/Closed Principle (O):** New functionality can be added by extending services rather than altering existing logic.
- **Liskov Substitution (L):** Services adhere to well-defined interfaces, allowing them to be swapped without breaking the system.
- **Interface Segregation (I):** APIs are designed to expose only necessary operations, minimizing unnecessary complexity.
- **Dependency Inversion (D):** High-level services rely on abstractions such as API contracts and event formats rather than concrete implementations.

Adherence to DRY and KISS

- **Don't Repeat Yourself (DRY):**

- Shared modules (e.g., for authentication, logging) are used across services to eliminate redundant code.
 - Standardized event structures promote reuse and consistency.
- **Keep It Simple, Stupid (KISS):**
 - Each microservice focuses solely on its core responsibility.
 - Event formats are designed to be clear and easy to interpret for all consuming services.